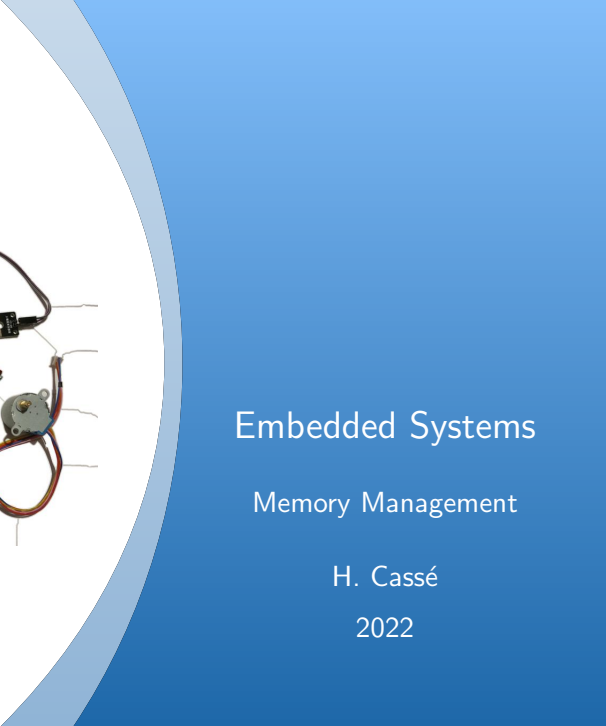




Memory Management

2022

Introduction

Embedded system memories:

- ▶ linear address space
- ▶ contains ROM, RAM, I/O
- ▶ no or very few protection
- ▶ often no virtual memory

Control code/data placement:

- ▶ binary format – ELF
- ▶ linker script – ld

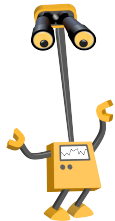
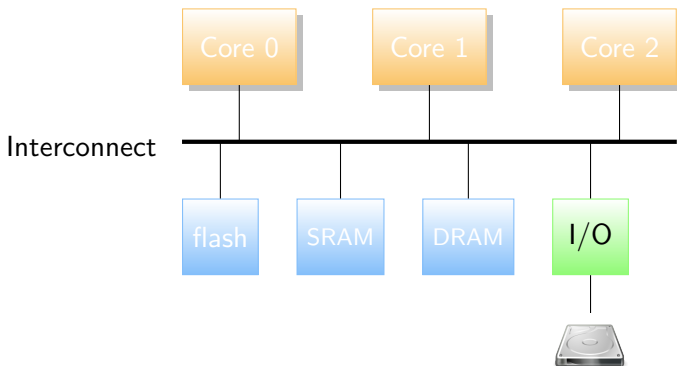


Outline

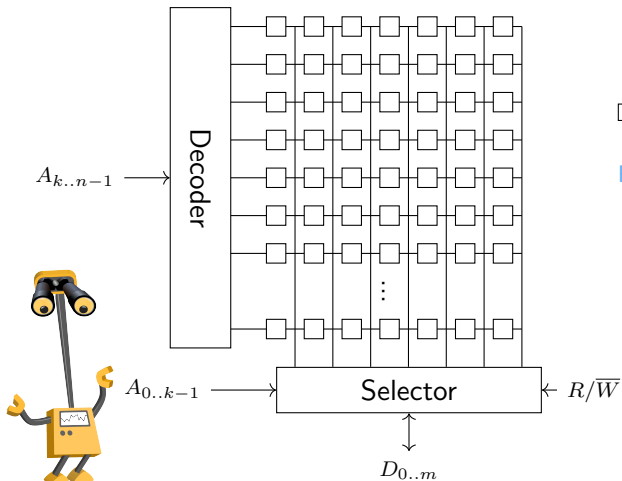
- 1 Introduction
- 2 Memory hierarchy
- 3 Binary Format
- 4 Linker Scripts
- 5 Conclusion



Modern microcontroller/System-On-Chip



Memory Structure



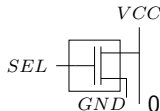
□ Memory cell.

Read ($R/\overline{W} = 1$)

- 1 when $CS = 1$
- 2 row $A_{k..n-1}$ copied to Selector
- 3 word $A_{0..k-1}$ selected
 $\Rightarrow D_{0..m}$
- 4 $ACK \leftarrow 1$

Read Only Memory

- ▶ Mask ROM (programmed at built time) – NOR implementation



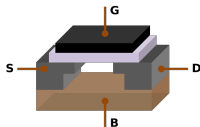
- ▶ Programmable ROM – one shot
based on fuse: cooled down $\Rightarrow$ 1, ok $\Rightarrow$ 0
- ▶ Erasable PROM – MOSFET + floating gate
Erased by UV exposition.
- ▶ Electrically EPROM
 - ▶ EEPROM – byte-grain writing, slow (parameters)
 - ▶ flash – page-grain writing, fast (code + data)



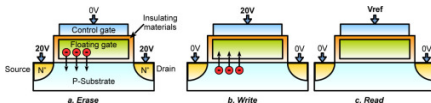
EEPROM

Electrically Erasable Programmable ROM

Field-effect transistor – state lost when power off



field-effect transistor with floating-gate (preserving state)



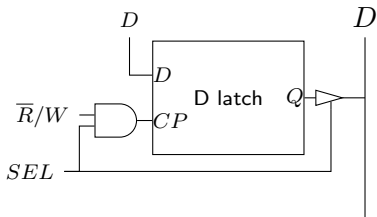
EEPROM (field electron emission), flash (hot-carrier injection)



SRAM

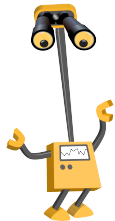
Static Random-Access Memory

cell = D latch (delay latch)



CP	D_t	Q_{t+1}
0	X	Q_t
1	0	0
1	1	1

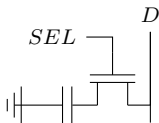
fast – same technology as microcontroller
expensive – ~ 20 transistors



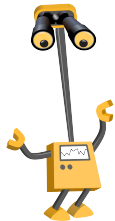
DRAM

Dynamic Random-Access Memory

cell = transistor + capacitor



cost = 1 transistor + 1 capacitor
speed = slow, variable
technology $\neq$ processor



read

- 1 set *SEL* to 1
- 2 capacitor emptied to *D*
- 3 bit $\Rightarrow$ bus
- 4 capacitor reloaded from selector

write

- 1 set *SEL* to 1
- 2 empty the capacitor
- 3 record bit from *D*

leakage $\Rightarrow$ need for refresh
the capacitor regularly $\Rightarrow$
slow down read/write.

To sum up

Type	t_{read}	t_{write}	Cost	Use
EEPROM	fast	slow	cheap	config.
flash	slow	very slow	cheap	firmware/code
SRAM	very fast	very fast	expensive	stack/global
DRAM	slow/irreg	slow/irreg.	very cheap	data

For DRAM:

t_{access} time to get/set word

t_{cycle} time before next operation

 $+ t_{refresh}$

EEPROM, flash, SRAM – in controller

DRAM – in a different die



Interconnect: bus (parallel)

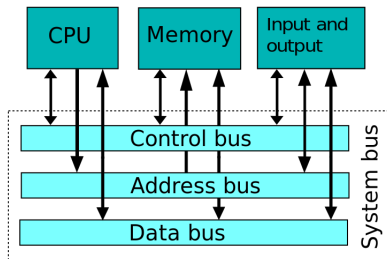
Properties:

- ▶ word size: 16b, 32b, 64b
- ▶ 1 bus $\leftrightarrow n$ cores
- ▶ 1 bus $\leftrightarrow n$ memory
- ▶ 1 request/time
- ▶ arbitration (round-robin, priority)

Work:

- 1 core acquire the bus
- 2 put the address (+ data – store)
- 3 wait for acknowledge (read data – load)
- 4 core release the bus

Memory hierarchy



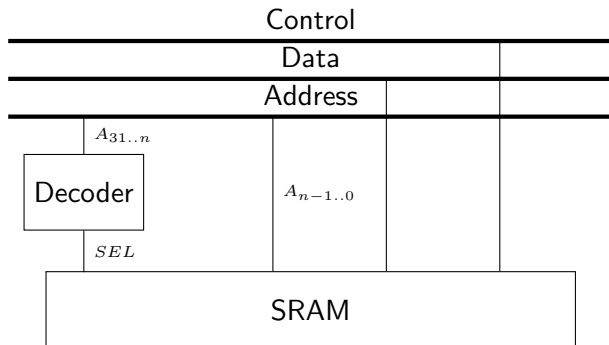
Control: CLK, DSEL, RD/WR, ACK

Access:

- ▶ single – 1 access $\rightarrow$ 1 word
- ▶ burst – 1 access $\rightarrow$ 1 block



How memories are connected?



Example: STM32 SRAM1

Address: $0x2000\ 0000-0x2001\ BFFF - 112\ KB\ n = 14\ (0xB = 0b1011)$

$SEL = \overline{A_{31}} \cdot \overline{A_{30}} \cdot \overline{A_{29}} \cdot \overline{A_{28}} \cdot \dots \cdot \overline{A_{16}} \cdot \overline{A_{15}} \cdot \overline{A_{14}}$

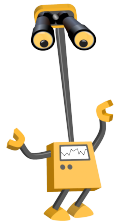
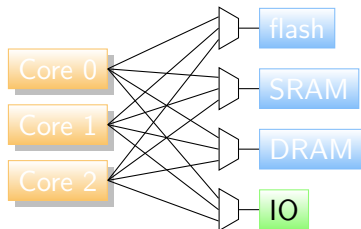
Interconnect: crossbar/matrix

Structure:

- ▶ masters: core
- ▶ slaves: memory
- ▶ 1 master $\rightarrow$ 1 slave
- ▶ arbitration if n cores $\rightarrow$ same slave

Work:

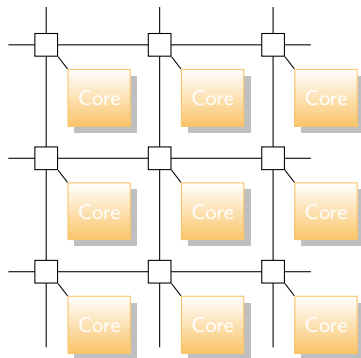
- 1 core require access to slave
- 2 for acquisition
- 3 put address (+ data)
- 4 wait for acknowledge
- 5 release the slave



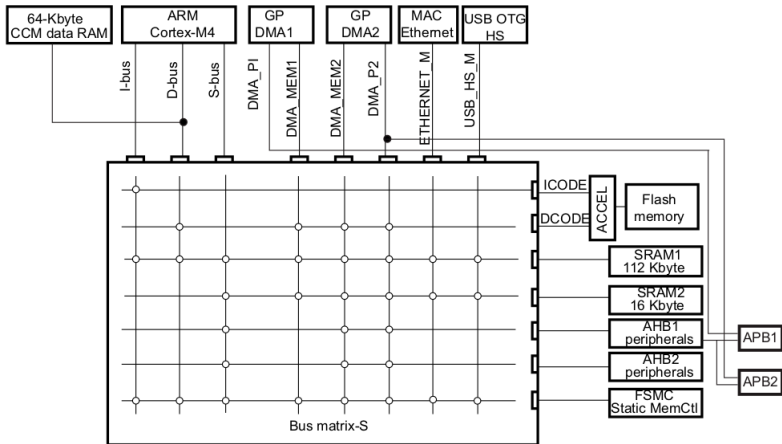
Interconnect: grid (close future)

Massively parallel machines:

- ▶ interconnect network (hyper/grid)
- ▶ 1 core $\leftrightarrow$ port
- ▶ memory in cores
- ▶ memory at some ports
- ▶ 1 access = 1 packet sent
- ▶ path in the grid



STM32 Example



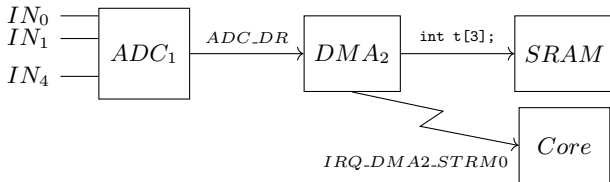
Direct Memory Access

 DMA_1, DMA_2

- ▶ 8 streams + priority
- ▶ connected to 8 channels
- ▶ data size: 1B, 2B, 4B
- ▶ increment
- ▶ FIFO 16B – direct/burst, reload threshold, circular
- ▶ $P \rightarrow M, M \rightarrow P, M \rightarrow M$ (DMA_1)



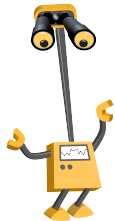
Example 1: getting ADC values



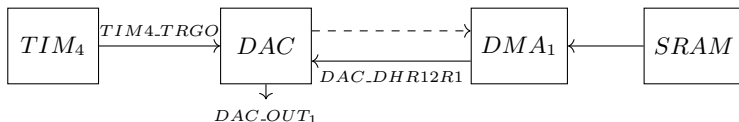
Connected to stream 0, channel 0 of DMA_2 :

```
DMA2_S0PAR = (uint32_t)&ADC2_DR;
DMA2_S0M0AR = (uint32_t)t;
DMA2_S0NDTR = 3;
DMA2_S0CR = DMA_CHAN0 | DMA_MSIZE_WORD | DMA_PSIZE_WORD | DMA_MINC
            | DMA_DIR_P2M | DMA_PFCTRL | DMA_EN | DMA_TCIE;
DMA2_LIFCR = DMA_TCIF0;
```

```
ADC_SQR1 = 3 << 20;
ADC_SQR3 = (0 << 0) | (1 << 5) | (4 << 10);
ADC_CR2 &= ~ADC_DMA; /* set 1: enable */
ADC_CR2 |= ADC_DMA;
ADC_CR1 = ADC_SWSTART; /* start ADC */
```



Example 2: playing values on the DAC



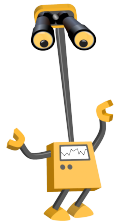
DAC1 connected to *DMA1*, stream 5, channel 7:

```
DMA1_S5PAR = (uint32_t)&DAC_DHR12R1;  
DMA1_S5M0AR = (uint32_t)t;  
DMA1_S5NDTR = 1024;  
DMA1_S5CR = DMA_CHAN7 | DMA_MSIZE_WORD | DMA_PSIZE_WORD | DMA_MINC  
            | DMA_DIR_M2P | DMA_PFCTRL | DMA_EN | DMA_MBURST_4;  
DMA1_S5FCR = DMA_SFCR_DMDIS | DMA_SFCR_FTHFULL;  
  
DAC_CR = DAC_DMAEN1 | DAC_TEN1 | DAC_EN1 | DAC_TSEL_TRGO4;
```

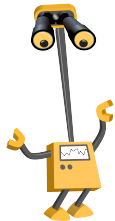
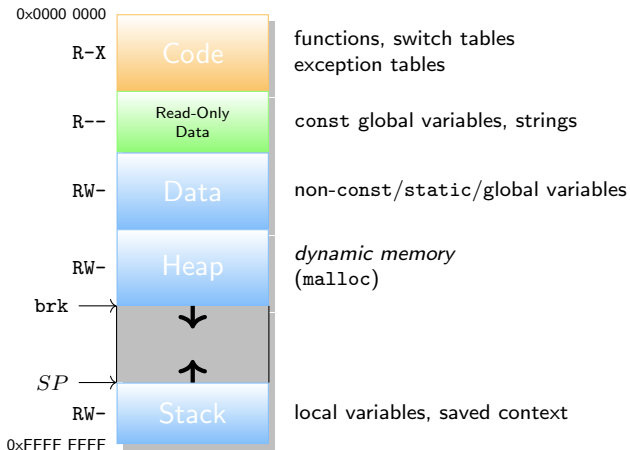


Outline

- 1 Introduction
- 2 Memory hierarchy
- 3 Binary Format
- 4 Linker Scripts
- 5 Conclusion



Memory Map Model



Executable and Linking Format

Objectives (basically for Unix System-V):

- ▶ executable fast load,
- ▶ adapted to OS
- ▶ multi-architecture support
- ▶ static & dynamic linking
- ▶ kernel + embedded code

Alternatives:

- ▶ COFF (Common Object File Format) – PE-COFF (Windows)
- ▶ universal binary (*fat binary*) – Apple
- ▶ .class Java byte-code



Execution
header
program table
segment 1
segment 2
...
section table

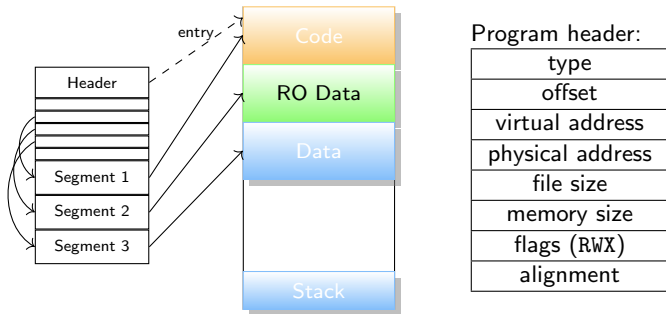
◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ≡ ≡ ↺ 🔍 ↻

Example

```
$ readelf -h ex1.elf
Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
Class:                               ELF32
Data:                                   2's complement, little endian
Version: .....1 (current)
OS/ABI: .....UNIX - System V
ABI version: .....0
Type: .....EXEC (executable)
Machine: .....ARM
Version: .....0x1
Entry point address: ...0x8000415
Program header offset: ...52 (bytes in the file)
Section header offset: ...176212 (bytes in the file)
Flags: .....0x5000400, Version 5 - EABI, hard-float ABI
Header size: .....52 (bytes)
Program header size: ...32 (octets)
Program header number: ...2
Section header size: ...40 (octets)
Section header number: ...19
String table index: ....18
....
```



Executing a program



For data, file size $\leq$ memory size:

- ▶ file size: initialized data (stored in file)
- ▶ memory size - file size: uninitialized data (set to 0)



Example

```
$ readelf -l ex1.elf
```

ELF file type is EXEC (executable)

Entry point 0x8000415

There are 2 program headers at offset 52

Program headers:

Type	Offset	Virt Addr	Phys. Addr	Fi.Size	Me.Size	Flags	Alignment
LOAD	0x010000	0x08000000	0x08000000	0x00c90	0x00c90	R X	0x10000
LOAD	0x020000	0x20000000	0x08000c90	0x00000	0x00408	RW	0x10000

Matching section/segment:

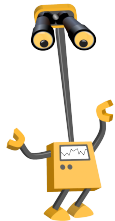
Sections of segment...

00	.text
01	.data .bss .stack



Example: Linux program – x86_64

Type	Offset File size	Virt. Addr Memory size	Phys. Addr Flags Alignment
PHDR	0x0000000000000040 0x00000000000002d8	0x0000000000000040 0x00000000000002d8	0x0000000000000040 R 0x8
INTERP	0x0000000000000318 0x000000000000001c	0x0000000000000318 0x000000000000001c	0x0000000000000318 R 0x1
[Interpreter: /lib64/ld-linux-x86-64.so.2]			
LOAD	0x0000000000000000 0x00000000000001cf8	0x0000000000000000 0x00000000000001cf8	0x0000000000000000 R 0x1000
LOAD	0x0000000000000200 0x00000000000009d1	0x0000000000000200 0x00000000000009d1	0x0000000000000200 R X 0x1000
LOAD	0x000000000000c000 0x00000000000040e0	0x000000000000c000 0x00000000000040e0	0x000000000000c000 R 0x1000
LOAD	0x0000000000010970 0x0000000000000710	0x0000000000010970 0x00000000000008d0	0x0000000000010970 RW 0x1000
DYNAMIC	0x0000000000010b98 0x00000000000001f0	0x0000000000010b98 0x00000000000001f0	0x0000000000010b98 RW 0x8
NOTE	0x0000000000000338 0x0000000000000020	0x0000000000000338 0x0000000000000020	0x0000000000000338 R 0x8
NOTE	0x0000000000000358 0x0000000000000044	0x0000000000000358 0x0000000000000044	0x0000000000000358 R 0x4
GNU_PROPERTY	0x0000000000000338 0x0000000000000020	0x0000000000000338 0x0000000000000020	0x0000000000000338 R 0x8
GNU_EH_FRAME	0x000000000000e070 0x0000000000000544	0x000000000000e070 0x0000000000000544	0x000000000000e070 R 0x4
GNU_STACK	0x0000000000000000 0x0000000000000000	0x0000000000000000 0x0000000000000000	0x0000000000000000 RW 0x10
GNU_RELRO	0x0000000000010970 0x0000000000000690	0x0000000000010970 0x0000000000000690	0x0000000000010970 R 0x1



Sections

Logical view:

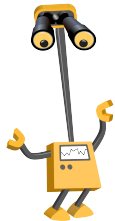
- ▶ `.text` – code
- ▶ `.rodata` – read-only data
- ▶ `.data` – initialized data
- ▶ `.bss` – uninitialized data
- ▶ `.strtab` – string table
- ▶ `.symtab` – symbol table
- ▶ `.rel.XXX` – relocation
- ▶ `.debug` – debugging

Section header:

name
type
flags
address
offset
size
link (section)
info
alignment
entry size

type: PROGBITS, SYMTAB, STRTAB.

```
flags: WRITE, ALLOC, EXECINSTR.
```



Example

```
$ readelf -S ex1.elf
```

There are 19 section headers, beginning at offset address 0x2b054:

Section header:

[Nr]	Nom	Type	Addr	Offset	Size	ES	Fl.	LN	Inf	Al
[0]		NULL	00000000	000000	000000	00		0	0	0
[1]	.text	PROGBITS	08000000	010000	000c90	00	AX	0	0	4
[2]	.data	PROGBITS	20000000	020000	000000	00	WA	0	0	1
[3]	.bss	NOBITS	20000000	020000	000008	00	WA	0	0	4
[4]	.stack	NOBITS	20000008	020000	000400	00	WA	0	0	1
[5]	.debug_info	PROGBITS	00000000	020000	001071	00		0	0	1
[6]	.debug_abbrev	PROGBITS	00000000	021071	0006e2	00		0	0	1
[7]	.debug_loc	PROGBITS	00000000	021753	00101b	00		0	0	1
[11]	.debug_line	PROGBITS	00000000	0241a0	000f2b	00		0	0	1
[12]	.debug_str	PROGBITS	00000000	0250cb	0055d3	01	MS	0	0	1
[13]	.comment	PROGBITS	00000000	02a69e	000059	01	MS	0	0	1
[14]	.ARM.attributes	ARM_ATTR	00000000	02a6f7	000034	00		0	0	1
[15]	.debug_frame	PROGBITS	00000000	02a72c	000258	00		0	0	4
[16]	.symtab	SYMTAB	00000000	02a984	000470	10		17	46	4
[17]	.strtab	STRTAB	00000000	02adf4	00019e	00		0	0	1
[18]	.shstrtab	STRTAB	00000000	02af92	0000bf	00		0	0	1



Building an object

file.c

```
#include <stdio.h>
...
printf("ok!");
...
x = errno;
...
```

stdio.h

```
...
printf(const char *, ...);
...
extern int errno;
...
```

CC -C

```
...
0x224 BL ???
...
0x330 ADR R3, ???;
...
```

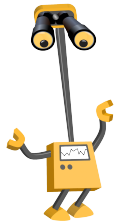
file.o/.symtab

```
main 296 FUNC GLOBAL DEFAULT 14
printf 0 NOTYPE GLOBAL DEFAULT UND
errno 0 NOTYPE GLOBAL DEFAULT UND
```

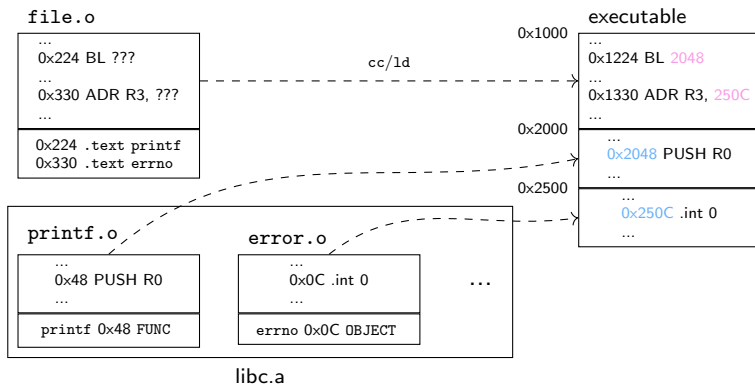
file.o/.rel.text

```
0x224 .text printf R_ARM_CALL 0
0x330 .text errno R_ARM_ABS32 0
```

file.o/.text



Linking into an executable



Static link edition: algorithm

Data: $\Omega = \{\text{objects}\}$, $L = \{\text{libraries}\}$

Result: $A : object \rightarrow address$: executable objects

$$a \leftarrow \text{base address}; A \leftarrow \{\} ;$$
while $\Omega \neq \emptyset$ **do**
$$\omega \leftarrow pop(\Omega); \Omega' \leftarrow \Omega' @ [\omega];$$
$$A[\omega] \leftarrow a; a \leftarrow a + size(\omega);$$
foreach $s \in symbols(\omega)$ **do**

if $\exists \omega' \in (\Omega \cup \Omega') \wedge s \in \omega'$ then

pass

else

if $\exists \omega' \in L \wedge s \in \omega'$ then

$$\Omega \leftarrow \Omega \cup \{\omega'\}$$

else

error: cannot resolve symbol s

end

end

end

end



Example

```
$ readelf -s ex1.o
```

Num:	Value	Type	Link	Scope	Ndx	Name
0:	00000000	0 NOTYPE	LOCAL	DEFAULT	UND	
...						
6:	00000000	0 NOTYPE	LOCAL	DEFAULT	13	\$d
7:	00000000	0 NOTYPE	LOCAL	DEFAULT	14	\$t
8:	00000104	0 NOTYPE	LOCAL	DEFAULT	14	\$d
...						
48:	00000001	296 FUNC	GLOBAL	DEFAULT	14	main
49:	00000000	0 NOTYPE	GLOBAL	DEFAULT	UND	tfp_printf

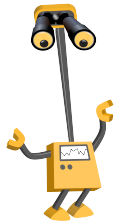
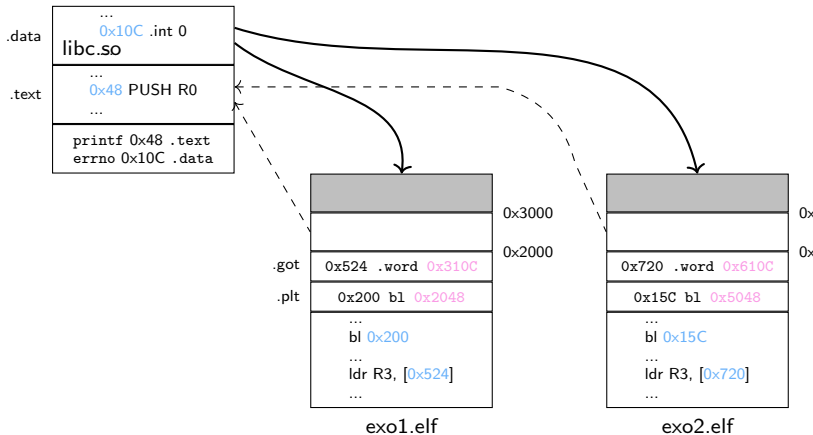
```
$ readelf -r ex1.o
```

Relocation section '.rel.text' at offset address 0x764c contains 6 entries:

Offset	Info	Type	Sym val.	Symbol name
00000008	0000310a	R_ARM_THM_CALL	00000000	tfp_printf
000000a6	0000310a	R_ARM_THM_CALL	00000000	tfp_printf
000000b0	0000310a	R_ARM_THM_CALL	00000000	tfp_printf
00000104	00000502	R_ARM_ABS32	00000000	.rodata.str1.4
0000010c	00000502	R_ARM_ABS32	00000000	.rodata.str1.4
00000120	00000502	R_ARM_ABS32	00000000	.rodata.str1.4



Example: libc.so



Controlling the code generation

Standard modifiers:

- ▶ `extern` – symbol defined in an external object,
- ▶ `local static` – local variable declared in `.data`,
- ▶ `global static` – symbol only visible in current object,
- ▶ `const` – put in `.rodata`.

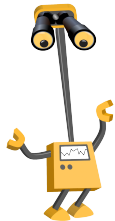
Non-standard (GCC):

- ▶ `__attribute__((aligned(size)))` – alignment
- ▶ `__attribute__((weak))` – weak linkage
- ▶ `__attribute__((section("SECTION")))` – section selection



Outline

- 1 Introduction
- 2 Memory hierarchy
- 3 Binary Format
- 4 Linker Scripts
- 5 Conclusion



link.ld

How it works?

How it works?

-

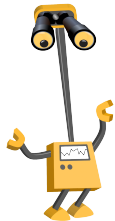
Memory Description

```
MEMORY {
    name(access): ORIGIN=address, LENGTH=size
}
```

access combination of r, w and x.

address address expression (C-like).

size size expression (C-like), N_K , N_M , ...



Commands: address assignment

$$identifier = expression ;$$

Also $+=$, $-=$, ...

identifier:

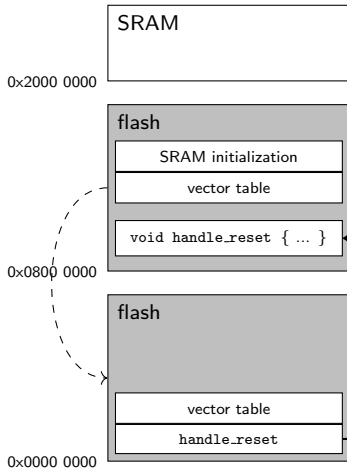
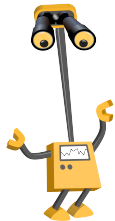
- ▶ Defined by assignment.
- ▶ Program symbol (read-only).
- ▶ ● – location counter
(incremented each time a section is added)

expression:

- ▶ C-style expressions,
- ▶ constants with K and M,
- ▶ `ALIGN(alignment)`, `ALIGN(address, alignment)`
- ▶ `ADDR(section)`, `SIZEOF(section)`, `ALIGNOF(section)`,
- ▶ `LENGTH(memory)`, `ORIGIN(memory)`
- ▶ `MAX(e1, e2)`, `MIN(e1, e2)`



Example: startup time



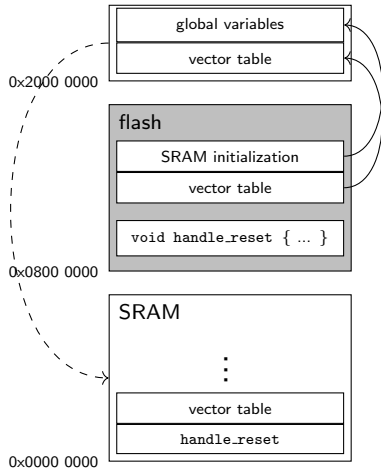
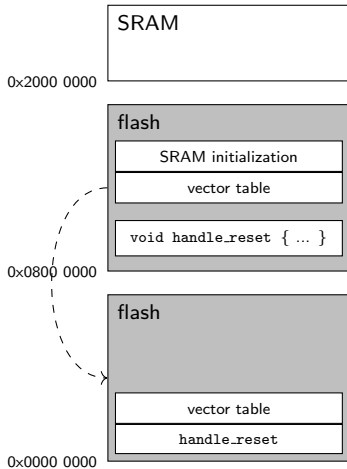
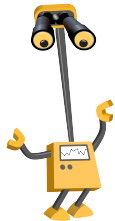
.data specification:

```
.data : AT(_data_flash)
{
    _vectab_in_ram = .;
    *(&.vector_table);
    _data_begin = .;
    *(&.data) *(&.data*)
    . = ALIGN(4);
    _data_end = .;
} > RAM
```

Memory alias remap:

```
SYSCFG.MEMRMP = 0b11;
```

Example: SRAM initialization



Example: SRAM initialization code

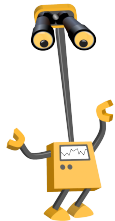
link.ld:

```
.data : AT(_data_flash)
{
    _vectab_in_ram = .;
    *(_vector_table);
    _data_begin = .;
    *(_data) *(_data*)
    . = ALIGN(4);
    _data_end = .;
} > RAM
```

Initialization of global variables:

```
extern uint32_t _data_flash;
extern uint32_t _data_begin;
extern uint32_t _data_end;
```

```
uint32_t *p = &_data_flash;
uint32_t *q = &_data_begin;
while (q < &_data_end)
    *q++ = *p++;
```



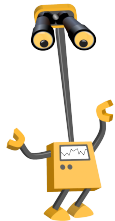
Exercise

This embedded application is made of modules that are represented by code, data and a specific datastructure declared in its own section `.mod`:

```
typedef struct {
    char name[16];
    uint32_t id;
    uint32_t version;
    void (*init)(struct module_t *m);
    void *data;
} module_t;
```

These section are grouped together in a section named `.mod`.

- 1 Write the linker script part that builds `.mod`.
- 2 Write the function `void init_modules()` that initializes the modules by calling the function pointer `init` of the data structure.



Outline

- 1 Introduction
- 2 Memory hierarchy
- 3 Binary Format
- 4 Linker Scripts
- 5 Conclusion

